Machine Learning Project Documentation

Model Refinement

1. Overview

Screen2Sleep is a proposed Android assistant designed to help users manage late-night phone use. Model A estimates the remaining duration of an ongoing app session and is the primary candidate for future on-device implementation. Model B explores the relationship between evening phone use and sleep duration, but its current results do not support a validated numerical sleep-loss warning. Therefore, Model B remains an exploratory research component rather than a confirmed application feature.

Where we start. In the previous stage (Data Preparation, Feature Engineering and Model Exploration), Model A was developed on the public LSApp dataset. After cleaning, it contains 275 users and 76,147 sessions. Each session still active at 1, 2, 5, 10, 15, 20 or 30 minutes was turned into a prediction snapshot (132,640 in total), described by 25 features that can be computed on the phone. The target was the remaining session time, and sessions with an unobserved end were treated as censored. Four survival models were compared (a Kaplan–Meier baseline, a Cox proportional-hazards model, a Weibull accelerated failure time (AFT) model and XGBoost with an AFT objective). The Weibull AFT was selected as the initial model for the app: its accuracy matched XGBoost, it was better calibrated, and its predictions can be computed using a closed-form formula suitable for Android implementation. For Model B, the GLOBEM 2018 cohort has been analysed using real participant-level data. However, the resulting mixed-effects model remains exploratory and has not demonstrated sufficient predictive improvement to support a numerical sleep-loss warning.

What this report covers. The Model Refinement part first evaluates the initial model and identifies its weaknesses (Section 2). It then describes the techniques used to address them (Section 3), the hyper-parameter tuning (Section 4), the cross-validation strategy (Section 5) and the feature selection (Section 6). The Test Submission part applies the refined model to held-out test data, reports the test metrics against the training and validation metrics, and describes how the model is deployed in the Android application.

Goals of the refinement. (1) Improve how well the model ranks and times session endings, especially for long sessions. (2) Keep the predicted probabilities well calibrated, because the app acts on them. (3) Keep the model small enough to run on the phone, using only features the app can compute.

2. Model Evaluation

2.1 Evaluation set-up

All models were evaluated with 5-fold cross-validation grouped by user: every snapshot of a user is either in the training folds or in the held-out fold, so the scores describe performance on people the model has never seen. Because Model A predicts a time rather than a class, survival metrics are used instead of accuracy, confusion matrices or ROC curves. The same metric definitions are considered in Test Metrics (TS 4). However, the initial evaluation and refinement experiments use different samples, landmark sets, and validation protocols. Their results are therefore reported separately and should not be interpreted as a direct before-and-after performance comparison.

- **C-index (Harrell):** the share of comparable pairs of snapshots that the model orders correctly by remaining time; 0.5 is random, 1 is perfect.
- **Integrated Brier score (IBS):** the squared error of the predicted survival curve, averaged over 0.5–30 minutes and weighted for censoring (IPCW); lower is better.
- **MAE and median absolute error:** the error of the predicted median remaining time, in minutes, on snapshots whose session end was observed.

2.2 Results of the initial models

Table 1. Five-fold user-grouped cross-validation of the initial models (mean ± standard deviation across folds).

Model	C-index ↑	IBS ↓	MAE (min) ↓	Median AE (min) ↓
Baseline (Kaplan–Meier)	0.539 ± 0.003	0.146 ± 0.006	6.57 ± 0.45	3.73 ± 0.07
Cox PH (stratified)	0.579 ± 0.011	0.142 ± 0.005	6.46 ± 0.43	3.64 ± 0.15
AFT (Weibull)	0.579 ± 0.012	0.142 ± 0.005	6.47 ± 0.42	3.76 ± 0.17
XGBoost (AFT)	0.583 ± 0.011	0.143 ± 0.006	6.48 ± 0.45	3.26 ± 0.18

All three covariate models outperform the Kaplan–Meier baseline on most evaluation metrics, although the improvements are modest. The initial Weibull AFT has a slightly higher median absolute error than the baseline (3.76 versus 3.73 minutes). The C-index rises from 0.54 to 0.58, the IBS falls from 0.146 to 0.142, and the MAE falls by only about 6 seconds. The three models are within one standard deviation of each other on C-index and IBS. XGBoost has the lowest median error (3.26 min versus 3.76 min for the Weibull AFT), which suggests it captures some non-linear effects that the linear models miss. The large gap between MAE (about 6.5 min) and median error (about 3.5 min) shows that a minority of very long sessions dominates the error.

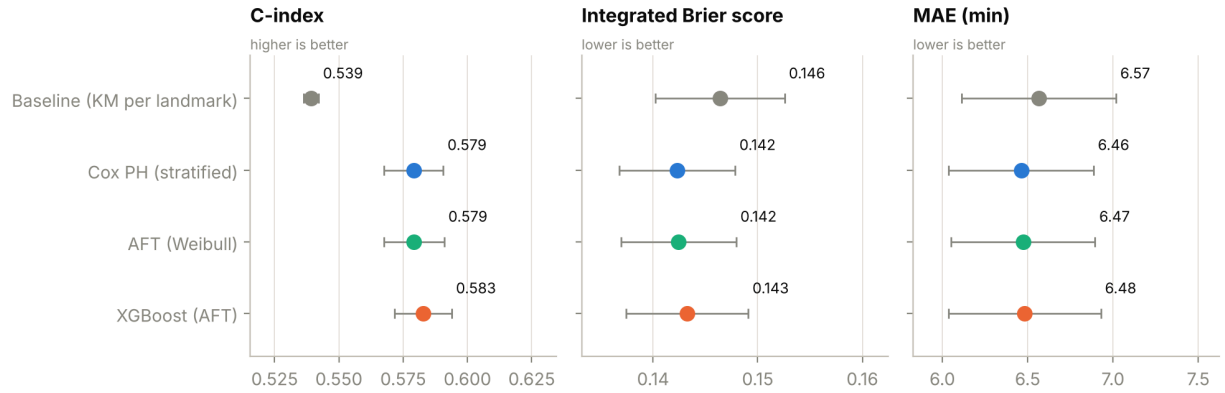


Figure 1. Cross-validated metrics of the initial models (mean ± SD over five user-grouped folds).

2.3 Calibration

For the app, the predicted probability that a session continues matters more than ranking alone, because it decides when a reminder is shown. At a 5-minute horizon the Weibull AFT lies close to the diagonal, while XGBoost systematically underestimates the probability that the session continues (Figure 2). Its lower median error therefore comes at the cost of probabilities that cannot be used directly.

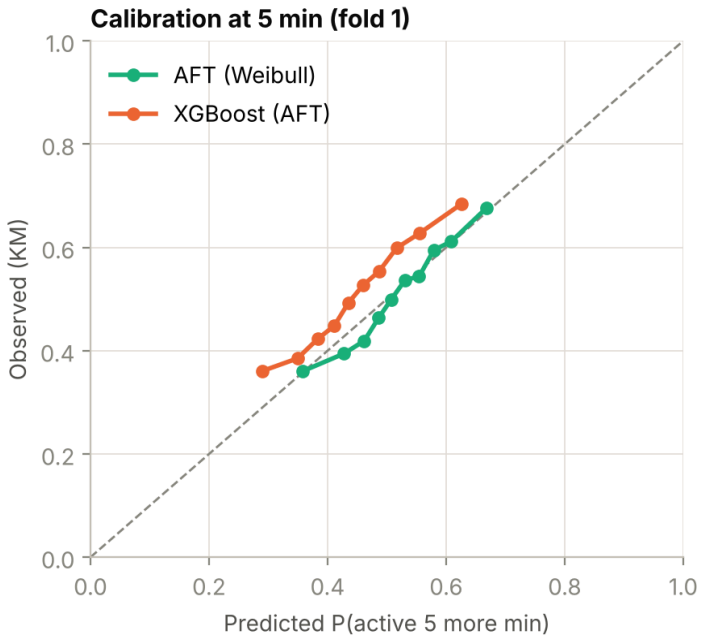


Figure 2. Calibration of P(session continues for 5 more minutes), held-out users of cross-validation fold 1, deciles of predicted probability.

2.4 Error by landmark

The error grows with the landmark (Figure 3): the longer a session has already lasted, the longer and more variable its remaining time, and the harder it is to predict. These late landmarks are exactly the long evening sessions that Screen2Sleep targets, so they are the main focus of refinement.

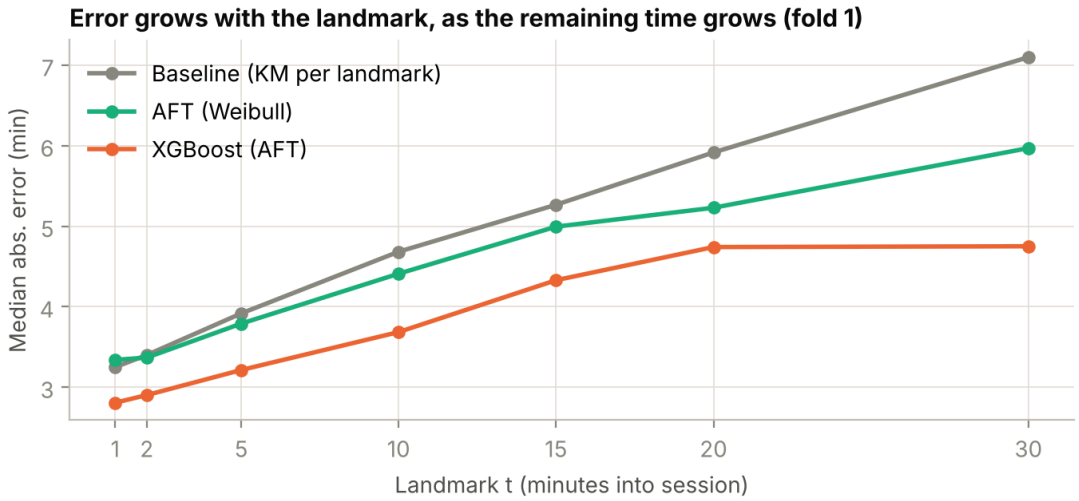


Figure 3. Median absolute error of the predicted remaining time by landmark (held-out users of cross-validation fold 1).

2.5 Interpretation of the initial Weibull AFT

The time ratios of the Weibull AFT are plausible. A rewards app in the foreground lengthens the remaining time by 29%, and a history of long sessions by 14% per standard deviation. Idle time (6% shorter per SD) and a long gap since the previous session (6%) shorten it. Several features have

time ratios very close to 1 (for example the mean past session length, 0.996, and the calls category, 1.014), which marks them as candidates for removal.

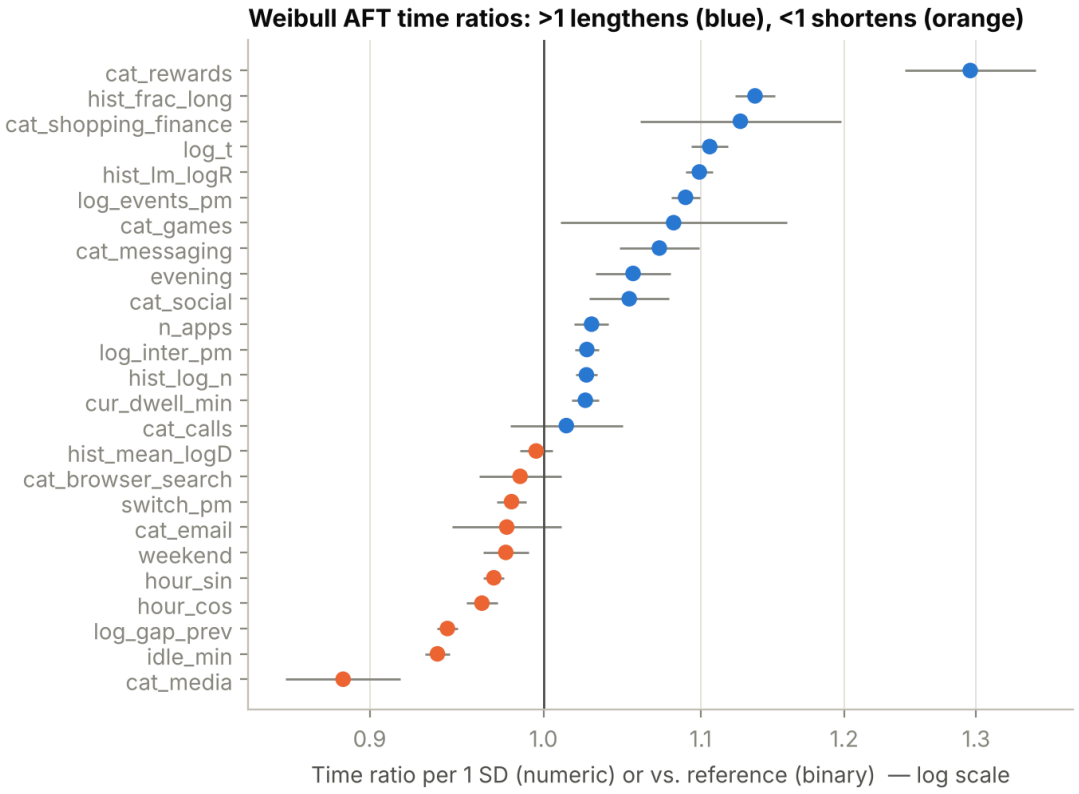


Figure 4. Time ratios of the initial Weibull AFT with 95% confidence intervals.

2.6 Weaknesses and refinement targets

The evaluation points to the following weaknesses, each addressed in a later section:

Table 2. Weaknesses of the initial model and where they are addressed.

Weakness	Evidence	Addressed in
Limited discrimination	C-index 0.58; small gain over the baseline	MR 3 Refinement Techniques, MR 4 Tuning
Large errors on long sessions	MAE ≈ 2 × median error; error grows with the landmark (Figure 3)	MR 3 Refinement Techniques
Non-linear model poorly calibrated	XGBoost underestimates continuation (Figure 2)	MR 3 (calibration), MR 4
Weak or redundant features	Several time ratios ≈ 1 (Figure 4)	MR 6 Feature Selection
Hyper-parameters chosen on one inner split	Tuning used cross-validation fold 1 only	MR 4 Tuning, MR 5 Cross-Validation
Training data differ from the app	LSApp sessions are split by a 5-min event gap, not by screen on/off; time zone assumed	TS 2 Data Preparation for Testing, TS 5 Deployment

Initial Evaluation of Model B (GLOBEM 2018)

Model B was initially developed to estimate the relationship between evening smartphone use and sleep-onset latency. However, 99.6% of the available non-missing sleep-onset observations were recorded as zero, making this outcome unsuitable for reliable continuous prediction. Therefore, Fitbit main-sleep duration was selected as an alternative exploratory target.

The initial evaluation used 4,718 eligible participant-nights from 124 participants in the GLOBEM 2018 cohort. A random-intercept linear mixed-effects model was fitted using three predictors: *use_within*, *use_between*, and *is_weekend*. The model was evaluated using a chronological split (3,725 training nights and 993 test nights), alongside five-fold participant-grouped cross-validation.

On the chronological held-out test set, the mixed-effects model achieved an MAE of 66.08 minutes and a median absolute error of 52.15 minutes, compared with 66.29 and 51.50 minutes for the personal-mean baseline, respectively. Although the mixed model reduced MAE by 0.21 minutes, the participant-cluster bootstrap confidence interval for the improvement (95% CI: −0.94 to +1.34 minutes) included zero, indicating no statistically reliable improvement over the baseline.

In participant-held-out cross-validation, the mixed model achieved a mean MAE of 68.24 minutes, compared with 69.41 minutes for the population-mean baseline. Its approximate 25–75% prediction interval achieved 48.4% empirical coverage.

The estimated within-person association was +0.689 minutes of main-sleep duration per additional ten minutes of evening smartphone use (95% CI: +0.327 to +1.052). Because the dataset is observational, this association cannot be interpreted as a causal effect.

Overall, the initial evaluation showed that Model B did not reliably outperform the personal-mean baseline in the primary held-out comparison. Consequently, the model remains exploratory and does not support a validated numerical sleep-loss warning. Subsequent feature-selection experiments (MR Section 6) retained the original mixed-effects specification without identifying a more accurate alternative.

3. Refinement Techniques

To address the weaknesses identified during the initial evaluation—specifically limited discrimination (C-index 0.58), large errors on long sessions (MAE ≈ 2 × median error), poor probability calibration in tree-based models, and a mismatch between LSApp session definitions and mobile operating system behavior—four targeted refinement techniques were implemented while preserving the on-device deployment constraint:

1. Personal Gamma-Frailty Offset for Model A (Empirical Bayes Adaptation): In the initial pooled Weibull AFT model, users with habitually long evening sessions and users with brief check-ins shared a single population scale parameter $\lambda = \exp(\beta_0 + \beta^T X)$. To account for unobserved user-level heterogeneity without sacrificing closed-form inference on Android, a multiplicative gamma-frailty offset $\theta \sim \text{Gamma}(a, a)$ was introduced. For each user, the personal multiplier updates dynamically from their completed prior sessions as $\theta = (a + n_events) / (a + \Sigma(R / \lambda)^{\rho})$. This scales the predicted quantiles via $r_q = \lambda \cdot (-\ln(1 - q) / \theta)^{(1}$

/ ρ), while naturally defaulting to the population model (θ = 1) for cold-start users with no prior history.

2. App-Level Session Reconstruction & Landmark Re-alignment: In the exploration phase, LSApp sessions were segmented using a 5-minute event gap, which did not reflect actual screen-on/screen-off boundaries on Android. During refinement, sessions were reconstructed per app by pairing Opened and Closed events, merging re-entries within 30 seconds, retaining unmatched sessions as right-censored, and aligning evaluation to five active decision landmarks (t ∈ {5, 10, 15, 20, 30} minutes; 12,566 snapshots in total). Furthermore, personalized historical predictors—most notably the user's own past mean log-duration for the same foreground app (hist_app_mean_logD)—were engineered to improve late-landmark timing.
3. Algorithm Selection for Native Probability Calibration: Although XGBoost (AFT) achieved a lower median error in initial cross-validation (3.26 min vs. 3.76 min for Weibull AFT), Figure 2 showed that it systematically underestimated continuation probabilities, making it unsuitable for threshold-based reminder triggers without heavy post-hoc calibration. Retaining the Weibull AFT backbone augmented with the personal gamma-frailty offset preserved natural probability calibration—achieving 50.8% empirical coverage for the nominal 25–75% interquartile interval on returning users—and is suitable for lightweight closed-form implementation in Kotlin.
4. Within/Between Person Decomposition for Model B: For Model B, initial exploration on the GLOBEM 2018 cohort revealed that 99.6% of non-missing sleep-onset latency observations were zero, preventing continuous latency modeling. The target was therefore reframed to Fitbit main-sleep duration, and evening screen exposure was decomposed into person-centered within-person deviations (use_within) and stable between-person norms (use_between) within a random-intercept linear mixed model (target ~ use_within + use_between + is_weekend) estimated via restricted maximum likelihood (REML).

4. Hyperparameter Tuning

In the exploration phase, hyperparameter choices relied solely on cross-validation Fold 1, risking optimistic bias. During refinement, all tuning decisions were isolated inside the inner chronological validation split (L_fit = 7,766 snapshots; L_val = 2,028 snapshots) defined in Section 5, leaving the outer test split untouched until final evaluation. Candidate configurations were compared using person-level cluster bootstrap confidence intervals (2,000 user resamples) to prevent row-level correlation from understating uncertainty.

Model A Hyperparameters & Marginal Likelihood Estimation: • L2 Regularization Penalty (penalizer): Evaluated across a logarithmic grid {0.001, 0.01, 0.1, 0.5, 1.0} in WeibullAFTFitter(penalizer=..., l1_ratio=0.0) to stabilize coefficient estimates and prevent variance inflation on rare 30-minute snapshots. • Gamma-Frailty Prior Shape (a): Rather than fixing a arbitrarily, the prior shape parameter of the Gamma(a, a) frailty distribution was estimated via maximum marginal likelihood (fit_prior_shape) over the training users by minimizing the negative marginal log-likelihood $-\Sigma\,[\ln\Gamma(a + D_i) - \ln\Gamma(a) + a\ln(a) - (a + D_i)\ln(a + E_i)]$, bounded over $\ln(a) \in [-5, 10]$. Sensitivity checks were also performed across fixed prior values $a \in \{10, 20, 35.2, 50\}$ on L_val to verify robustness against cold-start shrinkage. • Weibull Shape Parameter (ρ): Estimated jointly during maximum likelihood fitting ($\rho \approx 0.87 < 1$, confirming a decreasing hazard rate the longer a session remains active).

Model B Optimization & Estimation Settings: • For the linear mixed model, REML estimation with the Powell optimizer (method="powell") was selected over standard L-BFGS to guarantee reliable variance-component convergence across participant random intercepts (û_j). During feature-set comparisons in Section 6, maximum likelihood (reml=False) was paired with REML so that Akaike Information Criterion (AIC) values remained valid across varying fixed-effect specifications.

Insights Gained & Performance Impact:

- Minimal Regularization Optimal (penalizer = 0.001): Light L2 regularization (penalizer = 0.001) preserved the signal of informative predictors such as hist_app_mean_logD and log_elapsed_min without over-shrinking category effects, achieving an inner-validation all-landmark MAE of 3.594 min and C-index of 0.627. Stronger penalties (≥ 0.1) attenuated the personal app-history slope and slightly degraded validation C-index.
- Data-Driven Frailty Calibration (a = 35.2): Marginal likelihood optimization yielded a = 35.2 on the chronological split (and 28.7–36.2 across the five user-held-out folds). This prior strength struck an effective balance: returning users benefited from personalized adaptation—reducing 5-minute chronological test MAE from 3.29 min (personal-remaining baseline) to 3.18 min (difference -0.11 min, 95% CI [-0.20, -0.05]) and raising C-index from 0.562 to 0.580—while preventing noisy single-session fluctuations from destabilizing predictions for users with short histories.

The five-minute MAE, C-index, and interval coverage comparisons reported above originate from the earlier Task-4 pilot evaluation and are provided as supporting context rather than as a matched improvement estimate for the final 18-feature model.

5. Cross-Validation

The cross-validation design from the model exploration phase was kept as the backbone for both models, and three changes were added during refinement: an inner validation split for every refinement decision, cluster bootstrap intervals for model comparisons, and, for Model B, the removal of the leave-one-cohort-out plan. This section explains why the backbone was kept and why each change was made.

Why not ordinary random k-fold

Both datasets contain repeated observations of the same people over time. In LSApp, one app session produces up to five landmark snapshots (at 5, 10, 15, 20 and 30 minutes), and one user contributes hundreds of sessions; in GLOBEM, one participant contributes many nights. A random k-fold split would place snapshots of the same session, and later nights of the same person, in both training and test folds. The model would then be evaluated partly on information it has already seen, and on the past rather than the future, which gives optimistic errors. Every split in this project is therefore grouped (by session and by person) and, where it matters, ordered in time.

Validation design after refinement

Table 5.1. *Validation splits used in model refinement (snapshots for Model A, participant-nights for Model B).*

Split	Model A (LSApp)	Model B (GLOBEM 2018)	Purpose
Outer chronological 80/20 per person	9,794 train / 2,772 test	3,725 train / 993 test	Final test for returning users; used once per final comparison
Outer person-held-out 5-fold (GroupKFold)	Users split into 5 groups	124 participants in 5 groups	Performance for new users (cold start)
New: inner chronological validation inside the training part	7,766 fit / 2,028 validation	2,926 fit / 799 validation	All refinement decisions (feature selection, alternatives, tuning)
New: cluster bootstrap of paired differences	2,000 resamples of users	2,000 resamples of participants	Uncertainty of differences between models
Leave-one-cohort-out	Not applicable	**Dropped**	Only one cohort is analysed

Change 1: inner validation split

In the exploration phase, models were compared directly on the test split. That is acceptable for a single pre-planned comparison, but refinement involves many choices: which features to keep, which alternative to adopt, which hyperparameters to use. Making these choices on the test split would tune the model to the test data and make the final test result optimistic. Refinement decisions are therefore made on an inner split inside the training data, built with the same rule as the outer split: the last 20% of each person's training sessions (Model A) or training nights (Model B) by time. The outer test split is touched only once, when the final candidate is compared with the original model. All quantities learned from data are fitted inside the inner fitting part: Model A's fill values, means and standard deviations, baseline tables and frailty prior, and Model B's person-level exposure norms. The same inner split is available for the hyperparameter search in Section 4, so that no tuning decision uses the test data.

Change 2: cluster bootstrap intervals for model differences

Differences between the best candidates turned out to be small at the main five-minute landmark (about 0.1 minutes for Model A and 0.2 minutes for Model B), so a single MAE difference could not show whether one model is really better. Confidence intervals that resample individual rows would be too narrow, because observations of the same person are correlated. Paired differences are therefore resampled at the person level (2,000 resamples of users or participants); for the user-held-out evaluation, predictions from all five folds are pooled, since every user is in exactly one test fold.

This change altered the conclusions of the evaluation. For Model A, the advantage of the Weibull AFT with personal offset over the strong personal-remaining baseline is reliable for returning users (−0.11 min at t = 5, 95% interval −0.20 to −0.05) but not demonstrated for new users (−0.01 min,

interval −0.04 to +0.05). For Model B, the 0.21-minute improvement of the mixed model over the personal-mean baseline has an interval of −0.94 to +1.34 minutes, so it is not a reliable improvement. Without the bootstrap, both comparisons would have been reported as wins.

Change 3 (Model B): leave-one-cohort-out removed

The original plan for Model B was to hold out one GLOBEM cohort (study year) at a time, to test whether the relationship transfers between years. The reported analysis uses the 2018 cohort only, so this design cannot be run and the cohort term was removed from the model. Transfer across years remains future work for a multi-cohort analysis; the person-held-out 5-fold split is the closest available test of generalisation.

Leakage safeguards inside every split

For Model A, history features use only sessions that had ended before the prediction moment, and only completed (uncensored) sessions; these rules are checked by automated tests in the repository (ml/tests/test_features.py). For Model B, a participant's usual evening use is computed from that person's training nights only and reused unchanged for later nights; unseen participants receive the mean of the training participants' norms and a personal random intercept of zero. Eligibility filters for Model B (plausible ranges, bedtime at or after midnight, at least seven eligible nights) are applied before splitting, because they define the study sample rather than being learned from outcomes.

6. Feature Selection

Feature selection was applied to both models with the same protocol: decisions on the inner validation split only (Section 5), uncertainty from the person-level bootstrap, and a single comparison of the selected set with the original set on the test split. Because both models are small (9 feature groups for Model A, 5 candidate terms for Model B) and each fit takes seconds, a **wrapper** approach was used, in which each candidate set is refitted and evaluated directly. Filter methods such as correlation ranking were not used, because they ignore censoring (Model A) and the within/between-person structure (Model B).

Features excluded by design

Some features were excluded before any data-driven selection, for reasons of validity or deployment:

- **Model A:** raw app identity (87 levels) is replaced by 10 app categories, to avoid sparse levels and to transfer to apps that are not in LSApp; no content features are used, in line with the project's privacy rules; sequence models over the event stream are left out because they cannot be reproduced cheaply on the phone.
- **Model B:** night-segment phone use (00:00–06:00) is excluded, because use after going to bed can be a consequence of not falling asleep as well as a cause (reverse causation); GLOBEM's normalised features are not used, because the phone needs a formula in plain minutes; the cohort indicator is not identifiable with one cohort.

Model A: ablation and backward elimination

Two wrapper methods were applied to the Weibull AFT with personal offset. In **drop-one ablation**, each feature group was removed in turn and the model refitted; the change in validation MAE (all landmarks) shows how much that group contributes on top of the others (Table 6.1, Figure 6.1). In **backward elimination**, the group whose removal lowered validation MAE most was removed, and this was repeated until no removal helped.

Table 6.1. *Model A drop-one ablation on the inner validation split (all landmarks; full model MAE 3.59 min, C-index 0.627).*

Removed feature group	Validation MAE (min)	Change vs full, 95% CI (min)	C-index
hist_app_mean_logD	3.68	**+0.09 [+0.02, +0.19]**	0.618
log_elapsed_min	3.65	+0.05 [−0.01, +0.15]	0.623
category (9 dummies)	3.61	+0.01 [−0.02, +0.05]	0.625
log_gap_prev_min	3.60	+0.00 [−0.00, +0.01]	0.627
hist_lm_logR	3.59	−0.00 [−0.04, +0.03]	0.626
log_use_today_min	3.59	−0.00 [−0.00, +0.00]	0.627
is_weekend	3.59	−0.00 [−0.01, +0.00]	0.627
hist_log_n	3.59	−0.01 [−0.03, +0.01]	0.627
hour (sin, cos)	3.58	−0.01 [−0.04, +0.01]	0.629

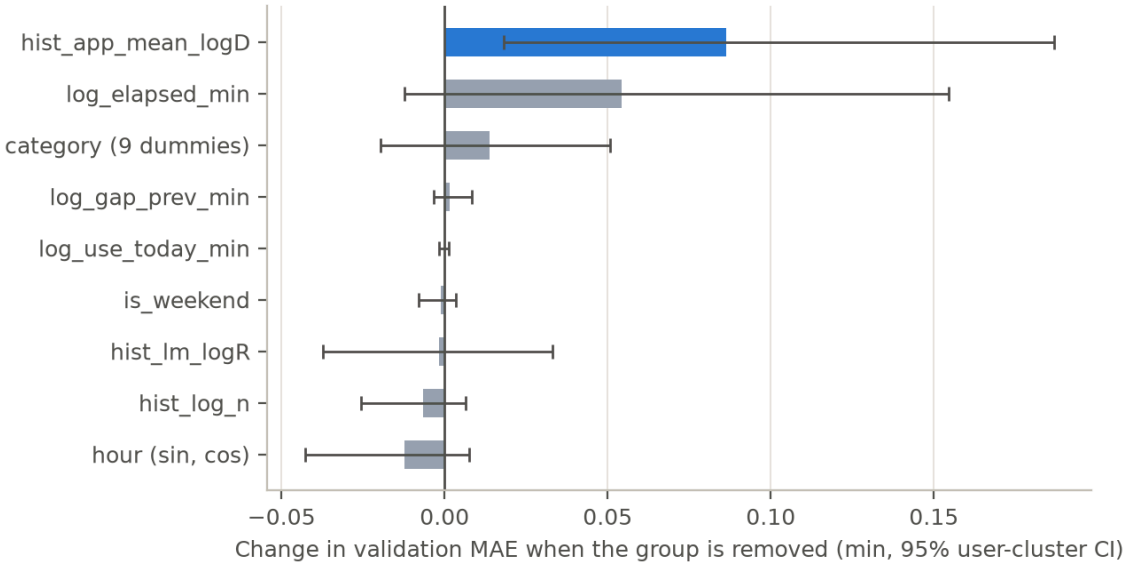


Figure 6.1. *Change in Model A validation MAE when each feature group is removed, with 95% user-cluster bootstrap intervals; the blue bar is the only interval that excludes zero.*

Only the user's past mean session length for the same app (hist_app_mean_logD) is clearly needed: removing it raises MAE by 0.09 minutes and lowers the C-index from 0.627 to 0.618. Elapsed time and app category also help, but their intervals include zero. Removing hour of day, use so far today, the history count or the weekend flag does not hurt, and for hour of day validation MAE even improves slightly. Hour of day may look relevant for an evening warning, but its effect is already captured by app category and the user's own history.

Backward elimination removed four groups in this order, each step lowering validation MAE: hour of day (3.594 → 3.582 min), use so far today (→ 3.579), the history count (→ 3.574) and the weekend flag (→ 3.572). The selected set keeps 13 of the 18 features: elapsed time, gap since the previous session, the two personal history features and the nine category dummies.

Table 6.2. *Model A full and selected feature sets on the test splits (one comparison; MAE in minutes).*

Feature set	Chronological MAE t = 5	Chronological MAE all	Chronological C-index	User-held-out MAE t = 5	User-held-out MAE all
Full (18 features)	3.178	4.185	0.621	2.964	3.951
Selected (13 features)	3.174	4.176	0.621	2.962	3.942

Effect on performance. On the test split, the selected set performs on par with the full set: the MAE difference over all landmarks is −0.01 minutes (95% interval −0.03 to +0.01), and coverage of the 25–75% interval is unchanged (0.51 vs 0.51). Feature selection therefore does not improve accuracy, but it gives a simpler model with the same accuracy. On the phone the selected set no longer needs to track the day's total use or the time of day, which simplifies the implementation and reduces the data the app has to keep. The reduced set is recommended for the next parameter export; the test metrics reported elsewhere in this document use the full set.

Model B: comparison of candidate terms

Starting from the reported random-intercept model (use_within, use_between, is_weekend), terms were removed or added one at a time. Each set was compared by validation MAE and by the Akaike information criterion (AIC). AIC was computed from maximum-likelihood fits, because restricted maximum likelihood (REML) values are not comparable between models with different fixed effects; predictions use the REML fits.

Table 6.3. *Model B candidate feature sets on the inner validation split (reported model MAE 60.37 min).*

Feature set	Validation MAE (min)	Change vs reported, 95% CI (min)	AIC (ML)
Reported model: within + between + weekend	60.37	—	33,765.7
Drop is_weekend	61.71	+1.34 [−0.14, +2.69]	33,946.7
Drop use_between	60.38	+0.00 [−0.01, +0.02]	33,763.9
Add evening unlock count (within-person)	60.41	**+0.04 [+0.01, +0.07]**	33,767.6
Add platform (Android)	60.37	−0.00 [−0.01, +0.01]	33,767.7
Intercept only (no exposure, no weekend)	61.97	**+1.60 [+0.11, +2.99]**	33,950.8

The weekend flag is the strongest term: removing it raises AIC by 181 points and validation MAE by 1.3 minutes. The model with only a personal intercept is clearly worse, so the fixed terms carry real information. The evening unlock count, proposed as a sensitivity feature for frequent checking, does not improve the model: its coefficient is −0.24 sleep minutes per 10 additional unlocks (95% CI −1.67 to +1.20), AIC rises by 1.9 and validation MAE becomes slightly worse. Platform does not improve the model either (coefficient −2.5 minutes, 95% CI −31.5 to +26.5); with only 10 Android participants in the eligible sample it cannot be estimated reliably, and no Android-specific estimate is reported. Removing use_between makes no practical difference (AIC −1.8, MAE +0.00), but the term is kept, because it separates stable between-person differences from the within-person association β_W, which is the quantity the warning would rely on.

Effect on performance. No candidate set improves on the reported model, so the reported model is retained unchanged. Feature selection therefore cannot close the gap identified in the evaluation: the mixed model does not beat the personal-mean baseline reliably, and the remaining limitation lies in the outcome and exposure measures rather than in the choice of features.

Summary

For both models, data-driven selection confirms that a small number of personal and calendar features carry most of the predictive information: the user's own app history for Model A and the weekend flag for Model B. For Model A, selection yields a simpler 13-feature model with unchanged

test accuracy. For Model B, it confirms the reported specification, and rejects the unlock-count and platform terms.

Test Submission

1. Overview

The test submission phase applies the final refined models once to data that played no part in training or refinement, and produces the predictions from which the test metrics (Section 4) are computed. All refinement decisions (feature selection, alternatives and tuning) were made on the inner validation split described in MR Section 5, so the outer test data were used only for the final evaluation.

Final models. Two models are submitted for testing:

- **Model A (LSApp):** the Weibull accelerated failure time (AFT) model with a personal gamma-frailty offset, using the full set of 18 features. This is the team's reported final refinement configuration. The 13-feature set from MR Section 6 is an evaluated alternative, not the reported model.
- **Model B (GLOBEM 2018):** the random-intercept linear mixed model with the terms use_within, use_between and is_weekend, which MR Section 6 retained unchanged. Model B predicts Fitbit main-sleep duration and remains an exploratory analysis. It does not predict sleep-onset delay.

Data scope. Two Model A experiments with different data scopes appear in this report and are kept apart. The initial-model comparison in MR Section 2 used 132,640 snapshots at seven landmarks (t = 1, 2, 5, 10, 15, 20 and 30 minutes) from 275 users, with 25 features and five-fold user-grouped cross-validation, and did not use a separate held-out test set. The refinement and test experiments use the 12,566-snapshot set at five landmarks (t = 5, 10, 15, 20 and 30 minutes) built in the previous phase, with the 18-feature model. The two experiments also define sessions differently: MR Section 2 uses the session identifiers supplied with LSApp, whereas the refinement set uses app sessions reconstructed per app (Section 2.1). Because the samples, session definitions, landmarks and validation designs differ, the test results are not presented as a before/after comparison with MR Section 2.

Steps. The test phase had five steps:

1. Freeze the final model specifications after refinement.
2. Prepare the test data with the same pipeline as the training data, fitting every learned quantity on training data only (Section 2).
3. Apply the fitted models to the test data and save the predicted quantiles and survival curves (Section 3).
4. Compute the test metrics and compare them with the validation results where these are available (Section 4).
5. Describe the deployment path to the Android application (Section 5).

The code for these steps is given in Section 6.

Two test settings. Each model is tested in two settings that answer different questions:

- A **chronological** split tests returning users: the model predicts a person's later sessions or nights from their earlier ones, as it would on the phone.

- A **person-held-out** five-fold split tests new users: the people in the test fold are never seen during training (cold start).

2. Data Preparation for Testing

The test data were prepared with exactly the same pipeline as the training data. The only difference is that no quantity is learned from them. This section lists the preparation steps and the safeguards against leakage from the test data into the model.

2.1 Model A (LSApp)

Same cleaning and feature pipeline. Test sessions pass through the cleaning rules fixed in the data preparation phase:

- Only consecutive exact duplicate rows are removed.
- Opened and Closed events are paired per phone session and app.
- Sessions without a Closed event are kept as right-censored.
- Re-entries to the same app within 30 seconds are merged.
- The same −5-hour local-time convention is applied.

Apps are mapped to the same 10 categories. Every session still open at t = 5, 10, 15, 20 or 30 minutes produces one snapshot, with target R = duration − t and the session's censoring flag. This gives 12,566 snapshots (9,337, 1,981, 738, 370 and 140 per landmark; 6.3% censored).

Split at session level. All landmark snapshots of one session are always placed in the same split, so a session cannot appear in both training and test data. Table TS2.1 summarises the resulting test sets.

- **Chronological split:** the first 80% of each user's sessions by start time form the training part and the last 20% the test part. This yields 9,794 training and 2,772 test snapshots.
- **Inner validation split:** the inner split (7,766 fit and 2,028 validation snapshots) lies entirely inside the 9,794 training snapshots. It was used only for refinement decisions.
- **Person-held-out split:** users are divided into five groups with GroupKFold on user_id. Each group serves once as the test set.

Table TS2.1. *Test data used in the test submission phase.*

	Model A (LSApp, 12,566 snapshots)	**Model B (GLOBEM 2018, 4,718 participant-nights)**
Chronological training part	9,794 snapshots (inner split 7,766 fit / 2,028 validation)	3,725 nights (inner split 2,926 fit / 799 validation)
Chronological test part	2,772 snapshots, last 20% of each user's sessions	993 nights, last 20% of each participant's nights
Person-held-out test	5 GroupKFold folds of users	5 GroupKFold folds of the 124 participants
Censored observations	Kept; used by C-index and IBS	Not applicable (no censoring)

Quantities fitted on training data only. Every quantity learned from data is estimated in the training part of each split and then applied unchanged to the test part:

- **Fill values:** the training-set means of history features that are undefined for a user's first sessions. A new user therefore starts at the training population average.
- **Standardisation:** the means and standard deviations used to z-score the numeric features. The binary weekend flag and the nine category dummies are not scaled.
- **Model parameters:** the baseline lookup tables, the Weibull AFT coefficients and the gamma-frailty prior.
- **Constant columns:** columns that are constant in a training split are dropped before fitting.

For the integrated Brier score, the inverse-probability-of-censoring weights are estimated from the training data as well.

No look-ahead in features. History features for a test snapshot use only that user's sessions that had ended before the prediction moment, and only completed (uncensored) ones. "Use so far today" resets at local midnight and counts only earlier sessions of the same day. These rules are checked by automated tests in the repository (ml/tests/test_features.py).

Censored and long sessions. Censored test snapshots are not dropped:

- The C-index and IBS use censored snapshots directly.
- MAE, median absolute error and interval coverage are computed on snapshots whose session end was observed.

Very long sessions are not clipped, because they are the sessions the warning targets. Their test samples are small, however: in the chronological test part the error metrics rest on only 180 snapshots at t = 15 and 36 at t = 30, so landmark-level results there are uncertain. Reward apps are kept in the test data, as in training.

2.2 Model B (GLOBEM 2018)

Study sample defined before splitting. The eligibility filters define the study sample and do not learn anything from the outcomes, so they are applied before the split. Table TS2.2 shows how the sample was reduced. Missing exposure or outcome values are not imputed.

- Evening screen-unlock duration and main-sleep duration must both be observed.
- Evening use must lie between 0 and 360 minutes, and main-sleep duration between 180 and 840 minutes.
- The recorded bedtime must be at or after midnight (≥ 1,440 minutes), so that the whole 18:00–24:00 exposure window precedes sleep.
- Each participant must have at least seven eligible nights.

Table TS2.2. *Model B sample before the split (2018 cohort).*

Step	Participant-nights	Participants
Participant-day grid	14,260	155
Screen and sleep both observed	6,324	136
Plausible ranges (use 0–360 min; sleep 180–840 min)	6,122	136

Bedtime ≥ 1,440 min	4,759	136
At least seven eligible nights	4,718	124

Splits. The splits follow the same rules as for Model A:

- **Chronological split:** each participant's nights are sorted by stored date. The first 80% form the training part (3,725 nights) and the rest the test part (993 nights).
- **Person-held-out split:** five GroupKFold folds of the 124 participants.

The seven-night rule counts all eligible nights, so two participants have fewer than seven training nights.

Training-only person norms. A participant's usual evening use (person norm) is computed from that person's training nights only. It is then reused unchanged for their test nights. The use_within and use_between terms are derived from this norm in 10-minute units. In the person-held-out folds, unseen participants have no norm, so they receive the mean of the training participants' norms.

Excluded inputs. Night-segment use (00:00–06:00) is excluded because of possible reverse causation. GLOBEM's normalised features are not used, because the phone needs a formula in plain minutes.

Confidentiality. Under the PhysioNet credentialed licence, the GLOBEM files are processed only on the credentialed team member's computer. Only aggregate metrics and model parameters leave that computer; no participant-level test records do.

3. Model Application

3.1 Single use of the test data

The final specifications were frozen after the refinement decisions on the inner validation split. Each model was then fitted on the training part of the outer split and applied once to the corresponding test part.

- **Chronological setting:** the models were fitted on the 9,794 training snapshots (Model A) and the 3,725 training nights (Model B).
- **Person-held-out setting:** the whole pipeline, including scalers, fill values and the frailty prior, was refitted in each of the five training folds and applied to the held-out users of that fold.

For the single feature-set comparison reported in MR Section 6, the 13-feature alternative was applied to the same test data in the same way.

3.2 Applying Model A

For every test snapshot the fitted model produces a predicted survival curve and three quantiles of the remaining time (Q25, median and Q75), in four steps:

1. **Transform the features.** The snapshot's 18 features are transformed with the training fill values, means and standard deviations.

2. **Compute the population scale.** The scale is $\lambda = \exp(\beta_0 + \beta^T x)$. The Weibull shape ρ is shared by all snapshots; its value of about 0.87 in the exploration fit is below 1, which means the hazard of the session ending decreases over time.
3. **Compute the personal offset.** The user's multiplier is $\theta = (a + n_events) / (a + \Sigma(R/\lambda)^\rho)$. The sum runs over the user's sessions that had ended before the prediction moment. The prior shape a is fitted on the training data (a = 35.2 on the chronological split; 28.7–36.2 across the user folds). A user with no history has $\theta = 1$, so the prediction equals the population model. This is always the case at the start of a person-held-out test user's data.
4. **Compute the predictions.** The quantiles are $r_q = \lambda \cdot (-\ln(1 - q) / \theta)^{(1/\rho)}$ for q = 0.25, 0.5 and 0.75, and the survival curve is $S(r) = \exp(-\theta\ (r/\lambda)^\rho)$. The survival curve is the input for the IBS on the 0.5–30-minute grid (Section 4).

The predicted median is the point prediction used for MAE and median absolute error. The Q25–Q75 range is used for interval coverage, and the survival curve for the IBS. This closed-form calculation is the same one the Android implementation is intended to reproduce in Kotlin (Section 5), so the test results describe the formula the phone is designed to use.

Python

```python
# personal.py: personal multiplier and quantiles for one test snapshot
def theta(a, n_events, expo):
    return (a + n_events) / (a + expo)        # = 1 for a user with no history
def quantiles(lam, rho, th, qs):
    # r_q = lambda * (-ln(1 - q) / theta)^(1/rho); theta = 1 gives the population model
    return np.column_stack([lam * (-np.log(1 - q) / th) ** (1 / rho) for q in qs])
```

3.3 Applying Model B

The prediction for a test night depends on the setting:

- **Chronological setting:** the prediction is the fixed part of the fitted model plus the participant's estimated random intercept: $\beta_0 + \beta_W \cdot use_within + \beta_B \cdot use_between + \beta_wk \cdot is_weekend + \hat{u}_j$. The use terms come from the training-period person norm.
- **Person-held-out setting:** an unseen participant has no estimated intercept, so the fixed part alone is used ($\hat{u}_j = 0$), with the mean training norm.
- **Baseline:** in both settings the model is compared with the personal-mean baseline, which predicts the mean outcome of the person's earlier nights (the population mean for a new person).
- **Interval:** an approximate 25–75% interval is formed as the prediction ± 0.674 times the estimated residual standard deviation. This plug-in interval does not include parameter or personal-effect uncertainty.

The within-person contrast $\beta_W \times \Delta use / 10$ is reported only as an observational association. It is not applied as a sleep-loss warning, in line with MR Section 6 and the deployment decision in Section 5.

3.4 Checks on the application code

As an additional check, the prediction code shown in Section 6 was run on synthetic data with known parameters. Table TS3.1 lists the checks, all of which passed. They confirm the following:

- The closed-form quantiles match the lifelines library.

- A new user receives the population prediction.
- The personal prior can be recovered from simulated data.
- Person norms are not affected by changes to test nights, so they do not leak test information.
- The Model B fit converges.

Table TS3.1. *Checks of the prediction code on synthetic data.*

Check	**Result**
Closed-form Q25, median and Q75 equal lifelines predict_percentile for the same fit	Passed (relative tolerance 10^{-6})
Columns that are constant in the training data are dropped before fitting	Passed
$\theta = 1$ with no history, and the personal quantiles with $\theta = 1$ equal the population quantiles	Passed
Gamma-prior shape is recovered from simulated users (true a = 35)	Passed (estimate 43.3, within the expected sampling range)
Chronological split puts the first 80% of each user's sessions in training and the rest later in time	Passed
Landmark snapshots and history features use earlier sessions only	Passed
Model B person norms do not change when test nights change	Passed
is_weekend marks Friday and Saturday nights	Passed
MixedLM with REML and the Powell optimiser converges and recovers the simulated within-person slope (true 0.70; estimate 0.72)	Passed
Model B personal offset is 0 for a new user and approaches the mean residual with many nights	Passed

These checks verify that the code computes what is described. They do not reproduce the reported LSApp or GLOBEM results, which come from the team's runs on the real data (Section 4).

4. Test Metrics

This section distinguishes three Model A analyses with different data scopes: (i) the initial-model comparison on the full LSApp evaluation set (132,640 snapshots, seven landmarks and

25 predictors), reported in MR Section 2; (ii) an earlier five-minute held-out analysis from Task 4; and (iii) the reported refinement analyses on a smaller 12,566-snapshot subset (five landmarks), using an 18-feature model and a 13-feature ablation candidate. The current reported refinement model is the 18-feature version, as confirmed by the team. The results are presented separately because the initial and refined experiments do not have an identical sample, landmark range or evaluation design. Model B remains an exploratory analysis of Fitbit main-sleep duration in the 2018 GLOBEM cohort.

4.1 Evaluation protocols and metrics

In the initial Model A experiment (MR Section 2), 275 LSApp users contributed 76,147 sessions and 132,640 snapshots at t = 1, 2, 5, 10, 15, 20 and 30 minutes. Five-fold cross-validation was grouped by user; each person belonged to one held-out fold, and MR Section 2 reports the mean ± standard deviation across folds rather than a separate final held-out test score. Refinement was subsequently conducted on the smaller 12,566-snapshot set at t = 5, 10, 15, 20 and 30 minutes. For this subset, MR Section 5 reports 9,794 chronological training and 2,772 test snapshots, with 7,766 fit and 2,028 inner-validation snapshots within the training portion. A separate five-fold user-held-out analysis examines generalisation to previously unseen people. Model B uses 4,718 eligible participant-nights from 124 people in the GLOBEM 2018 cohort, including 3,725 training and 993 chronological test nights and a 2,926/799 inner fit/validation split. These counts describe different experiments; the 12,566 refinement snapshots must not be described as the full 132,640-snapshot dataset.

For Model A, Harrell's concordance index (C-index; higher is better) evaluates discrimination under censoring. The integrated Brier score (IBS; lower is better) evaluates survival-probability error over 0.5–30 remaining minutes, using inverse-probability-of-censoring weights (IPCW) estimated from the training sample. Mean absolute error (MAE) and median absolute error (MedAE), expressed in minutes, evaluate predicted median remaining time only for observations with an observed session end. The empirical coverage of the predicted 25th–75th percentile interval is an additional calibration measure (nominal coverage: 50%). Classification accuracy, confusion matrices and ROC-AUC are not primary metrics for this survival-time task. The final 18-feature refinement IBS has not been supplied and must not be inferred from the initial-model IBS.

4.2 Initial Model A benchmarks: full-data cross-validation

MR Section 2 compares four initial models on the full 132,640-snapshot, seven-landmark dataset. The results below are five-fold user-grouped cross-validation means ± standard deviations. The initial Weibull AFT was chosen as the starting model because of its calibration and simple on-device formula; the comparison is a baseline study, not the final 18-feature model's held-out test.

Initial model	C-index ↑	IBS ↓	MAE (min) ↓	MedAE (min) ↓
Kaplan–Meier baseline	0.539 ± 0.003	0.146 ± 0.006	6.57 ± 0.45	3.73 ± 0.07
Cox PH (stratified)	0.579 ± 0.011	0.142 ± 0.005	6.46 ± 0.43	3.64 ± 0.15

Weibull AFT (initial)	0.579 ± 0.012	0.142 ± 0.005	6.47 ± 0.42	3.76 ± 0.17
XGBoost AFT	0.583 ± 0.011	0.143 ± 0.006	6.48 ± 0.45	3.26 ± 0.18

Table TS4.1. Initial Model A results on 132,640 snapshots and seven landmarks (MR Section 2; five-fold user-grouped CV, mean ± SD). Not a matched pre/post-refinement comparison.

4.3 Earlier Task-4 pilot: held-out results at the five-minute landmark

Table TS4.2 reports the earlier Task-4 five-minute results from the 12,566-snapshot exploratory setup. In the chronological held-out split, the reported Weibull AFT with personal offset produced MAE 3.18 minutes versus 3.29 minutes for the personal-remaining baseline, and C-index 0.580 versus 0.562. The central 25–75% interval covered 50.8% of observed outcomes. The paired five-minute MAE difference was −0.11 minutes (user-cluster bootstrap 95% CI −0.20 to −0.05) for returning users, whereas the observed difference for new users was −0.01 minutes (95% CI −0.04 to +0.05). These are previously reported pilot results; they are retained for context but are not the same evaluation as the full-data initial-model CV or necessarily a separate test of the final 18-feature export.

Evaluation / model	**MAE (min)**	**MedAE (min)**	**C-index**	**Q25–Q75 coverage**
Chronological, personal remaining baseline	3.29	1.44	0.562	45.8%
Chronological, Weibull AFT + personal offset	3.18	1.51	0.580	50.8%
User-held-out, personal remaining baseline	2.97	1.40	0.555	46.8%
User-held-out, Weibull AFT + personal offset	2.96	1.54	0.577	52.2%

Table TS4.2. Earlier Task-4 results at t = 5 minutes from the 12,566-snapshot pilot, reproduced for context; do not compare directly with Table TS4.1 as a matched refinement effect.

4.4 Refinement-stage feature-set comparison on the subset

MR Section 6 compares the reported 18-feature refinement model with a reduced 13-feature alternative on the 12,566-snapshot, five-landmark subset. The 18-feature specification is the team's current final refinement choice; the 13-feature version remains an ablation/selection candidate, not the

reported final model. Inner-validation all-landmark MAE is approximately 3.594 minutes for 18 features and 3.572 minutes for 13 features. Chronological held-out all-landmark MAE is 4.185 versus 4.176 minutes, respectively (difference −0.01 minutes, 95% CI −0.03 to +0.01), with reported 25–75% coverage of 0.51 for both. These results indicate essentially unchanged predictive accuracy under feature reduction; they do not establish an improvement over MR Section 2 because the initial experiment used 132,640 snapshots, seven landmarks and a different cross-validation design.

Model / outcome	Training MAE	**Inner validation MAE**	**Chronological test MAE**	**Person-held-out MAE**
Model A: 18 features, final reported refinement; all five landmarks	Not reported	3.594 min	4.185 min	3.951 min
Model A: 13 features, alternative; all five landmarks	Not reported	3.572 min	4.176 min	3.942 min
Model B mixed model; main-sleep duration	Not reported	60.37 min	66.08 min	68.24 min
Model B personal-mean baseline	Not reported	Not reported	66.29 min	69.41 min

Table TS4.3. Available MAE results from the 12,566-snapshot Model A refinement experiment (MR Section 6) and separate Model B analysis. Training values have not been provided. Inner validation and outer test use different observations; Model A and Model B are different prediction tasks. Do not treat this table as a matched initial-versus-refined comparison. The chronological C-index of the final 18-feature model is 0.621 across all five landmarks, as reported in MR Section 6. This value should not be confused with the five-minute C-index of 0.580 from the earlier Task-4 pilot evaluation.

The reported final refinement specification is Model A with 18 features; the smaller 13-feature set is an evaluated alternative. The supplied model_A_params.json represents an earlier 25-input feature schema (14 numerical and 11 binary fields), and no matching 18-feature Android export has been verified in the provided material. Consequently, the 18-feature model may be identified as the final reported refinement configuration, but not as a confirmed deployed model. Critically, the full-data initial Weibull AFT (132,640 snapshots; seven landmarks; five-fold grouped CV) and the 18-feature refinement model (12,566 snapshots; five landmarks; chronological/held-out tests) do not support a direct numerical before/after improvement claim. A rigorous performance difference would require both configurations to be evaluated on the same observations, landmarks and splits.

4.5 Model B: exploratory sleep-duration evaluation and limitations

Model B was originally intended to predict sleep-onset latency. In the available GLOBEM cohort, 7,443 of 7,476 nonmissing candidate sleep-onset observations are zero (99.6%), making this target unsuitable for the intended continuous prediction. The model was therefore evaluated on Fitbit main-sleep duration as a different exploratory outcome. On 993 chronological held-out nights, the personal-mean baseline yields MAE 66.29 minutes and MedAE 51.50 minutes; the random-intercept mixed model yields MAE 66.08 minutes and MedAE 52.15 minutes. The difference is too small to establish reliable superiority (paired improvement CI −0.94 to +1.34 minutes). On participant-held-out folds, the respective mean MAEs are 69.41 and 68.24 minutes. The mixed model's approximate central-interval coverage is 48.4%; this plug-in interval does not fully propagate all parameter or personal-effect uncertainty.

The fitted within-person association is +0.689 minutes of main-sleep duration per additional ten evening screen-unlock minutes (approximate 95% CI +0.327 to +1.052). This observational relationship cannot be interpreted causally or repurposed as the effect of an individual app session on time to fall asleep. The available model does not validate a numerical sleep-loss or sleep-onset-delay warning; any decision to implement Model B as a different feature requires further measurement and predictive validation.

4.6 Reporting limitations and metrics not available

Training-set MAE for the selected refinement configuration has not been provided in the available outputs, so Table TS4.3 identifies it as not reported rather than imputing a value. The same applies to a final held-out IBS for the 18-feature model: the available IBS of 0.142 ± 0.005 is the initial Weibull AFT five-fold CV result on the full dataset, not a final refinement test metric. A future IBS comparison should use the same 0.5–30-minute grid and training-derived IPCW estimator, and should also disclose the different sample/split designs. Final hyperparameter settings, a matching 18-feature export and training/validation/test outputs for all four requested survival metrics are not independently verifiable from the supplied document. No unsupported performance improvement is claimed.

5. Model Deployment

The intended deployment is a privacy-conscious Android assistant that estimates remaining foreground-app session time and offers user-controlled reminders without transmitting detailed usage histories to a server. In the current reporting, the 18-feature Weibull AFT is the selected refinement specification, evaluated on a 12,566-snapshot subset. An available JSON parameter file describes an older 25-input model rather than the selected 18-feature specification. The available materials therefore document model development and an export approach, but they do not independently establish that the selected 18-feature model has been exported to Kotlin, run in a working APK or deployed to users. Implemented, proposed and unverified capabilities are distinguished below.

5.1 Candidate runtime architecture

The Model A runtime design uses the closed-form Weibull accelerated failure-time prediction. Coefficients, preprocessing means and standard deviations, application-category mapping and the shape parameter can be serialized into versioned JSON, after which Kotlin can reproduce the same median remaining-time and 25th/75th percentile predictions from Android usage events. Android

UsageStatsManager and UsageEvents are proposed to supply foreground-application transitions. A personal offset/frailty component can adapt predictions from previously completed sessions. Earlier project documentation describes a Python model-export step and proposed Python–Kotlin parity checks; these statements describe the planned integration path, not verified parity for the selected 18-feature model.

For an Android implementation of the current refinement model, the exact 18-feature schema and transformation order must match the training code. Required checks include session reconstruction, handling of censored or incomplete sessions, local-time interpretation, application-category fallback, missing-history defaults and agreement of Q25, median and Q75 values between Python and Kotlin on identical test cases. The provided 25-input JSON cannot be used as evidence that the separate 18-feature model has already been deployed. No APK, automated parity-test results, measured latency, battery profile or user-pilot results for that version were verified in the supplied project material.

5.2 User experience, permissions and safeguards

The proposed pilot uses passive collection of usage history during the day and a bounded evening warning window beginning 60 minutes before the user's chosen target bedtime. The 60-minute window is an initial product parameter, not a learned optimum. The user can dismiss or mute a notification, turn warnings off, and delete stored data. The concept specifies that intervention notifications stop after the target bedtime, to avoid disturbing sleep. Android usage-access consent and, if applicable, runtime notification permission must be obtained through the operating system. Processing should remain on the device where possible, with only essential aggregate local state retained and no participant-level GLOBEM records distributed through the app.

A short optional morning diary and Health Connect sleep-session import are described as potential pilot data-collection mechanisms. They should be described as planned or under implementation unless actual Android code and permission flows are demonstrated. The diary labels are not interchangeable with the Fitbit main-sleep-duration target used for Model B training, so diary observations must not silently update the Model B random effect or support a sleep-loss estimate.

5.3 Restricted use of Model B

Current validation does not justify implementing Model B as a numerical sleep-onset-delay or sleep-loss predictor. The original sleep-onset variable is unusable for this target because of its extreme zero inflation; the exploratory mixed model instead predicts main-sleep duration and does not reliably outperform the personal-mean baseline in the primary held-out comparison. Accordingly, the defensible near-term product scope is a Model A remaining-session estimate plus a user-configured bedtime-overrun reminder, subject to implementation verification. Model B can remain in the report as exploratory research, while a separate team decision and additional data validation would be required before enabling any numerical sleep-effect feature.

5.4 Deployment verification and current status

Component	Supported status	Evidence / pending check

Prior Python fit/evaluation	Reported	MR Section 2; MR Sections 5–6 use a separate smaller subset
Model A parameter export	Older 25-input artifact available	No verified matching export for current 18-feature refinement model
Kotlin prediction engine	Not verified here	Request code path and Python/Kotlin parity tests
Android UsageStats event capture	Planned; implementation unverified	Request manifest, permission flow and device logs
Notifications, quiet hours and user controls	Specified in product design	Request tested UI/screenshots, behavior and permissions
Health Connect and morning diary	Exploratory; not validated for a numeric warning	Keep numerical sleep-loss claims out of the current product description
Model B numerical sleep warning	Not supported by current validation	Do not enable a numeric sleep-loss claim without suitable target data and external validation
APK/device or user pilot	No completion evidence supplied	Request demo version, test record and limitations

5.5 Unverified deployment evidence

The review materials do not include a confirmed build of the selected 18-feature Model A, a Python–Kotlin parity-test report or device logs demonstrating UsageStats collection and notification behaviour. The deployment status therefore remains “proposed or not independently verified” for those components. The implementation owner should provide code paths, a matching parameter export, test output or screen captures if these elements have since been completed. Until such evidence is available, they must not be described as successfully deployed. Measured runtime, memory and battery claims are likewise omitted because supporting benchmarks were not supplied.

6. Code Implementation

The code lives in the project repository:

- **Model A** is in ml/model_a/: sessions.py → features.py → train.py, with evaluate.py, personal.py and export.py. The automated leakage tests are in ml/tests/test_features.py.
- **Model B** is in ml/model_b/globem_model_b.py. It runs only on the credentialed team member's computer and writes aggregate tables, figures and parameters only.

The Model A run order from the repository root is:

1. python -m ml.model_a.train (evaluation tables)
2. python -m ml.model_a.export (parameter file and Python–Kotlin parity cases)
3. python -m ml.model_a.report_figures (figures)

The key sections are shown below, with comments explaining each step.

6.1 Session reconstruction and cleaning (sessions.py)

```python
def drop_consecutive_duplicates(events):
    # Rule 1: drop a row only if it is identical to the row directly before it
    same_as_prev = (events[COLUMNS] == events[COLUMNS].shift()).all(axis=1)
    return events[~same_as_prev].reset_index(drop=True)
def build_sessions(events):
    # Rules 2-3: pair Opened/Closed per (session_id, app_name); unmatched
    # sessions are kept as right-censored. Rule 4: merge re-entries within 30 s.
    s = merge_reentries(pair_events(drop_consecutive_duplicates(events)))
    s["duration_s"] = (s.end - s.start).dt.total_seconds()
    s["duration_min"] = s.duration_s / 60
    s["event"] = (~s.censored).astype(int)                  # 1 = end observed
    s["local_start"] = s.start + pd.Timedelta(hours=-5)     # Rule 5: time-zone convention
    return s.sort_values(["user_id", "start"]).reset_index(drop=True)
```

6.2 Landmark snapshots and history features (features.py)

```python
def add_session_history(s):
    ...
    # Completed sessions only (a censored duration is a lower bound); shift() so a
    # session never sees its own duration. Same-app sessions never overlap.
    s["hist_app_mean_logD"] = s.logD.where(s.event == 1).groupby(
        [s.user_id, s.app_name]).transform(lambda x: x.shift().expanding().mean())
    s["use_today_min"] = s.groupby([s.user_id, day]).duration_min.transform(
        lambda x: x.shift(fill_value=0).cumsum())
    s["use_today_min"] -= _running_overlap_min(s, day)  # minutes after this session's
start

def past_landmark_logR(L):
    # Only completed sessions that ENDED before the prediction moment start + t
    moment = (L.start + pd.to_timedelta(L.t, unit="m")).to_numpy()
    for idx in L.groupby(["user_id", "t"]).indices.values():
        done = L.iloc[idx].query("event == 1").sort_values("end")
        k = np.searchsorted(done.end.to_numpy(), moment[idx], side="left")
        csum = np.concatenate([[0], np.log(done.R.to_numpy()).cumsum()])
        out[idx] = np.where(k > 0, csum[k] / np.maximum(k, 1), np.nan)

for t in [5, 10, 15, 20, 30]:                       # landmark snapshots
    alive = s[s.duration_min > t].copy()            # still open at t minutes
    alive["t"] = t
    alive["R"] = alive.duration_min - t             # target: remaining minutes
```

6.3 Test split and training-only preprocessing (evaluate.py, train.py)

```python
def chronological_split(s, train_frac=0.8):
    # True for the first 80% of each user's sessions by start time;
    # all landmark snapshots of a session inherit the session's split
    rank = s.groupby("user_id").start.rank(method="first")
    n = s.groupby("user_id").start.transform("size")
    return rank <= train_frac * n
def fit_scaler(tr):
    # fitted on the training split only; applied unchanged to the test split
    num, _ = feature_names()
    fill = {c: float(tr[c].mean()) for c in HISTORY}   # new user -> population mean
    filled = tr[num].fillna(fill)
    return {"fill": fill, "mean": filled.mean().to_dict(),
            "std": filled.std().replace(0, 1).to_dict()}
def transform(d, scaler):
    num, binary = feature_names()                       # binary: is_weekend, cat_*
    x = d[num].fillna(scaler["fill"])
    x = (x - pd.Series(scaler["mean"])) / pd.Series(scaler["std"])   # z-score
    return pd.concat([x, d[binary]], axis=1)
```

6.4 Model A: fitting and applying to the test data (train.py, personal.py)

```python
def fit_weibull(X, d):
    X = X.loc[:, X.nunique() > 1]            # drop columns constant in this split
    return WeibullAFTFitter(penalizer=0.001).fit(
        X.assign(R=d.R.values, event=d.event.values), "R", "event")

def weibull_quantiles(lambda_coef, rho_log, X, qs=(0.25, 0.5, 0.75)):
    """r_q = lambda * (-ln(1 - q))^(1/rho); the same formula Kotlin uses."""
    lam = np.exp(lambda_coef["Intercept"] + sum(
        X[c].to_numpy() * b for c, b in lambda_coef.items() if c != "Intercept"))
    rho = np.exp(rho_log)
    return np.column_stack([lam * (-np.log(1 - q)) ** (1 / rho) for q in qs])

def exposure(R, lam, rho):
    return (R / lam) ** rho          # censored sessions count through exposure only

def fit_prior_shape(u):
    # Gamma(a, a) prior; a by maximum marginal likelihood over training users
    per_user = u.groupby("user_id")[["d", "e"]].sum()
    D, E = per_user.d.to_numpy(), per_user.e.to_numpy()
    def nll(log_a):
        a = np.exp(log_a)
        return -np.sum(gammaln(a + D) - gammaln(a) + a * np.log(a) - (a + D) * np.log(a +
E))
    return float(np.exp(minimize_scalar(nll, bounds=(-5, 10), method="bounded").x))

def theta(a, n_events, expo):
    return (a + n_events) / (a + expo)   # = 1 for a new user

def quantiles(lam, rho, th, qs):
```

```
    return np.column_stack([lam * (-np.log(1 - q) / th) ** (1 / rho) for q in qs])
```

6.5 Model A: test metrics (evaluate.py)

```
Python
def metrics(pred_q, te):
    R, E = te.R.to_numpy(), te.event.to_numpy()
    lo, med, hi = pred_q[:, 0], pred_q[:, 1], pred_q[:, 2]
    done = E == 1                                  # errors on completed sessions
    err = np.abs(med[done] - R[done])
    return dict(mae=err.mean(), medae=np.median(err),
                c_index=concordance_index(R, med, E),   # uses censored rows too
                coverage=np.mean((R[done] >= lo[done]) & (R[done] <= hi[done])),
                width=np.mean(hi - lo))
```

6.6 Model B: test preparation, fitting and personal offset (globem_model_b.py)

```
Python
def add_exposure_features(n, train_mask):
    # n: one row per participant-night (pid, date, evening_use_min, target)
    # person norms come from training nights only, so a test night never sees itself
    norm = n[train_mask].groupby("pid").evening_use_min.mean()
    grand = norm.mean()
    n["person_norm"] = n.pid.map(norm).fillna(grand)          # new person -> population
mean
    n["use_within"] = (n.evening_use_min - n.person_norm) / 10   # 10-min units
    n["use_between"] = (n.person_norm - grand) / 10
    n["is_weekend"] = n.date.dt.dayofweek.isin([4, 5]).astype(int)   # Fri/Sat night
    return n
def fit_model_b(train):
    # random intercept per person, REML with the Powell optimiser
    m = smf.mixedlm("target ~ use_within + use_between + is_weekend",
                    train, groups=train["pid"])
    return m.fit(reml=True, method="powell")
def personal_offset(residuals, sigma2, tau2):
    # normal-normal update; equals 0 (population model) for a new user
    n = len(residuals)
    u_hat = np.sum(residuals) / (n + sigma2 / tau2)
    u_var = 1 / (n / sigma2 + 1 / tau2)
    return u_hat, u_var
```

6.7 Initial model evaluation (mr2_model_evaluation.py, MR Section 2)

This script produces Table 1 and Figures 1–4 of MR Section 2. Imputation, standardisation and the censoring distribution used for the IPCW weights are fitted on the training folds only. The IBS

function was also checked on synthetic data: a constant survival probability of 0.5 without censoring gives exactly 0.25, and with censoring the IPCW-weighted score of the true model (0.0677) matches its score on the same data without censoring (0.0680).

Python

```
# mr2_model_evaluation.py: initial-model evaluation for MR Section 2 (Table 1)

LANDMARKS = [1, 2, 5, 10, 15, 20, 30]          # prediction points, minutes into a
session
GRID = np.linspace(0.5, 30, 60)                # horizon for the Brier score, minutes

def prep(tr, te):
    """Impute and standardise using the TRAINING fold only, then apply to the test
fold."""
    tr, te = tr.copy(), te.copy()
    for c in ["hist_mean_logD", "hist_frac_long", "hist_lm_logR", "log_gap_prev"]:
        fill = tr[c].mean()                    # a user's first session has no history
        tr[c] = tr[c].fillna(fill); te[c] = te[c].fillna(fill)
    mu, sd = tr[NUM].mean(), tr[NUM].std().replace(0, 1)
    tr[NUM] = (tr[NUM] - mu) / sd; te[NUM] = (te[NUM] - mu) / sd
    return tr, te, (mu, sd)

def ibs(S_pred, R, E, G):
    """Integrated Brier score with inverse-probability-of-censoring weights (Graf et al.,
1999).
    S_pred: n × len(GRID) predicted survival; G: Kaplan–Meier fit of the censoring
distribution."""
    Gt = np.clip(G.survival_function_at_times(R).values, 1e-3, None)
    Gg = np.clip(G.survival_function_at_times(GRID).values, 1e-3, None)
    bs = [np.mean(S_pred[:, j] ** 2 * ((R <= tau) & (E == 1)) / Gt
                  + (1 - S_pred[:, j]) ** 2 * (R > tau) / Gg[j]) for j, tau in
enumerate(GRID)]
    return trapz(bs, GRID) / (GRID[-1] - GRID[0])

def evaluate(name, med, S_pred, te, G):
    """C-index on the predicted median, IBS on the curve, MAE / median AE on observed
ends."""
    R, E = te.R.values, te.event.values
    med = np.clip(np.nan_to_num(med, nan=300, posinf=300), 0, 300)
    ev = E == 1
    return dict(model=name, c_index=concordance_index(R, med, E), ibs=ibs(S_pred, R, E,
G),
                mae=np.mean(np.abs(med[ev] - R[ev])), medae=np.median(np.abs(med[ev] -
R[ev])))

folds = list(GroupKFold(n_splits=5).split(L, groups=L.user_id))   # 5-fold, grouped by
user
for k, (tri, tei) in enumerate(folds):
    tr, te, _ = prep(L.iloc[tri], L.iloc[tei])
    G = KaplanMeierFitter().fit(tr.R, 1 - tr.event)          # censoring distribution
(IPCW)
    ...
    # Parametric AFT (log elapsed time is a covariate instead of strata)
    aft = AFT_CLS(penalizer=0.001).fit(tr[FEATS + ["R", "event"]], "R", "event")
    Sp = aft.predict_survival_function(te[FEATS], times=GRID).values.T
    aft_med = aft.predict_median(te[FEATS]).values
    rows.append(evaluate(f"AFT ({AFT_DIST})", aft_med, Sp, te, G) | {"fold": k})
```

6.8 Model refinement code

Hyperparameter search and marginal-likelihood frailty tuning on the inner validation split (ml/model_a/tune.py, MR Section 4). Every candidate penalty and frailty prior is fitted strictly on L_fit and scored on L_val using the exact preprocessing and closed-form prediction functions from Sections 6.3–6.4

```python
from scipy.optimize import minimize_scalar
from scipy.special import gammaln
def tune_penalizer_and_frailty(L_fit, L_val, feature_groups):
    """
    Tunes the Weibull AFT L2 penalizer and evaluates the marginal-likelihood
    gamma-frailty prior shape (a) strictly on the inner validation split (L_fit -> L_val).
    """
    scaler = fit_scaler(L_fit)
    cols = columns(feature_groups)
    X_fit = transform(L_fit, scaler)[cols]
    X_val = transform(L_val, scaler)[cols]
    penalizer_grid = [0.001, 0.01, 0.1, 0.5, 1.0]
    tuning_records = []

    for pen in penalizer_grid:
        # 1. Fit Weibull AFT on inner training split (L_fit)
        X_tr_clean = X_fit.loc[:, X_fit.nunique() > 1]
        aft = WeibullAFTFitter(penalizer=pen, l1_ratio=0.0).fit(
            X_tr_clean.assign(R=L_fit.R.values, event=L_fit.event.values),
            duration_col="R",
            event_col="event",
        )
        lam_coef, rho_log = weibull_params(aft)
        rho = np.exp(rho_log)

        # 2. Compute baseline scale & cumulative exposure on L_fit per user
        lam_fit = np.exp(
            lam_coef["Intercept"]
            + sum(X_fit[c].to_numpy() * b for c, b in lam_coef.items() if c !=
"Intercept")
        )
        fit_expo_df = pd.DataFrame({
            "user_id": L_fit["user_id"].values,
            "d": L_fit["event"].values,
            "e": exposure(L_fit["R"].values, lam_fit, rho),
        })
        # 3. Estimate gamma-frailty prior shape 'a' by marginal MLE on L_fit
        a_mle = fit_prior_shape(fit_expo_df)
        # 4. Apply fitted parameters & personal offset to inner validation split (L_val)
        pred_q, _ = personal_weibull_quantiles(
            lam_coef, rho_log, L_fit, X_fit, L_val, X_val, a_override=a_mle
        )
        # 5. Evaluate MAE (observed ends), C-index, and Q25-Q75 calibration coverage
        val_scores = metrics(pred_q, L_val)
        tuning_records.append({
            "penalizer": pen,
            "a_mle": round(a_mle, 2),
            "val_mae_all": val_scores["mae"],
            "val_medae_all": val_scores["medae"],
            "val_c_index": val_scores["c_index"],
```

```
            "val_coverage_50": val_scores["coverage"],
        })
    results_df = pd.DataFrame(tuning_records).sort_values(
        by=["val_mae_all", "val_c_index"], ascending=[True, False]
    )
    best_cfg = results_df.iloc[0].to_dict()
    return best_cfg, results_df
# Selected configuration on the 18-feature inner split:
# penalizer = 0.001, a_mle = 35.2 (chronological split), val_mae_all = 3.594 min
```

(a) Inner chronological validation split (ml/model_a/evaluate.py, ml/model_a/feature_selection.py).

The same rule as the outer test split is applied a second time inside the training part, so that feature selection and other refinement decisions never use the test data. Model B uses the same rule per participant.

Python

```
def chronological_split(s, train_frac=0.8):
    # True for the first 80% of each person's sessions (or nights) by start time
    rank = s.groupby("user_id").start.rank(method="first")
    n = s.groupby("user_id").start.transform("size")
    return rank <= train_frac * n

def split_by_sessions(L, s_part):
    # landmark snapshots follow their session, so one session never sits on both sides
    m = L.sid.isin(s_part.sid).to_numpy()
    return L[m].reset_index(drop=True), L[~m].reset_index(drop=True)

s_train = s[chronological_split(s)]                # outer split: the test part is
touched once
L_train, L_test = split_by_sessions(L, s_train)
s_inner = s_train[chronological_split(s_train)]    # same rule, applied inside the
training part only
L_fit, L_val = split_by_sessions(L_train, s_inner) # every refinement decision is scored
on L_val
(b) Person-level cluster bootstrap of paired differences, 2,000 resamples
(ml/model_a/evaluate.py, ml/model_a/train.py).
def bootstrap_mae_diff(pred_a, pred_b, te, n_boot=2000, seed=42):
    """MAE(a) - MAE(b) on completed sessions, with a paired 95% interval that resamples
users, not rows."""
    done = (te.event == 1).to_numpy()
    R = te.R.to_numpy()[done]
    err_a, err_b = np.abs(pred_a[done, 1] - R), np.abs(pred_b[done, 1] - R)
    codes, users = pd.factorize(te.user_id.to_numpy()[done])
    # per-user error sums: one bootstrap draw = one sample of whole users with all their
snapshots
    sum_a, sum_b, cnt = np.bincount(codes, err_a), np.bincount(codes, err_b),
np.bincount(codes)
    pick = np.random.default_rng(seed).integers(0, len(users), (n_boot, len(users)))
    diffs = (sum_a[pick].sum(1) - sum_b[pick].sum(1)) / cnt[pick].sum(1)
    lo, hi = np.quantile(diffs, [0.025, 0.975])
    return float(err_a.mean() - err_b.mean()), float(lo), float(hi)

# User-held-out evaluation: every user is in exactly one test fold,
# so the five folds' predictions are pooled into one test set before bootstrapping
```

```
pooled = {m: np.vstack([p[m] for p in fold_preds]) for m in fold_preds[0]}
```

(c) Model A: drop-one ablation and backward elimination (ml/model_a/feature_selection.py).

```python
def predict(groups, tr, te):
    # Weibull AFT + personal offset refitted on the chosen feature groups only;
    # scaler and frailty prior are fitted on tr, never on te
    scaler = fit_scaler(tr)
    cols = columns(groups)
    Xtr, Xte = transform(tr, scaler)[cols], transform(te, scaler)[cols]
    q, _ = personal_weibull_quantiles(*weibull_params(fit_weibull(Xtr, tr)), tr, Xtr, te,
Xte)
    return q
# Drop-one ablation: remove each feature group in turn and compare validation MAE with the
full model
full_q = predict(all_groups, L_fit, L_val)
for g in all_groups:
    q = predict([x for x in all_groups if x != g], L_fit, L_val)
    d, lo, hi = bootstrap_mae_diff(q, full_q, L_val)   # d > 0: the model is worse without
g
# Backward elimination: drop the group whose removal lowers validation MAE most, while any
removal helps
current, best = list(all_groups), score(full_q, L_val)["mae_all"]
while len(current) > 1:
    trials = {g: score(predict([x for x in current if x != g], L_fit, L_val), L_val) for g
in current}
    g, sc = min(trials.items(), key=lambda kv: kv[1]["mae_all"])

    if sc["mae_all"] >= best:

        break

    current.remove(g)

    best = sc["mae_all"]

# One final comparison of the selected and the full set on the outer test split

d, lo, hi = bootstrap_mae_diff(predict(current, L_train, L_test), predict(all_groups,
L_train, L_test), L_test)
```

(d) Model B: candidate-term comparison with ML-based AIC (local GLOBEM feature-selection script, aggregate output only).

```python
SETS = {
    "reported model":   "use_within + use_between + is_weekend",
    "drop is_weekend":  "use_within + use_between",
    "drop use_between": "use_within + is_weekend",
    "+ unlocks_within": "use_within + use_between + is_weekend + unlocks_within",
    "+ platform":       "use_within + use_between + is_weekend + android",
    "intercept only":   "1",
}
for name, rhs in SETS.items():
     # REML fit for prediction, as in the reported model
    r = smf.mixedlm(f"target ~ {rhs}", fit_part, groups=fit_part.pid).fit(reml=True,
method="powell")
```

```
    # ML fit for AIC: REML likelihoods are not comparable between different fixed effects
    rml = smf.mixedlm(f"target ~ {rhs}", fit_part, groups=fit_part.pid).fit(reml=False,
method="powell")
     aic = -2 * rml.llf + 2 * (len(rml.fe_params) + 2)   # + random-intercept and residual
variances
    val_mae, pred = mae(r, val)          # fixed effects + personal random intercept
#Only aggregate numbers (MAE, AIC, coefficients) are printed; no pid, date or row leaves
the script
```

Conclusion

This report refined and tested the two models behind Screen2Sleep. The initial evaluation of Model A (Section 2) showed that the three covariate survival models generally, though not uniformly, outperformed the Kaplan–Meier baseline. However, the improvements were modest: on 132,640 snapshots and five-fold user-grouped cross-validation, the C-index rose from 0.54 to 0.58 and the MAE fell from 6.57 to 6.47 minutes. The Weibull AFT was kept as the starting model because it was as accurate as XGBoost, better calibrated and simple enough to run on the phone.

Refinement and testing of Model A. The reported refinement configuration is a Weibull AFT model with a personal gamma-frailty offset and 18 features, evaluated on a 12,566-snapshot subset of reconstructed app sessions. All refinement decisions were made on an inner validation split, and model differences were assessed using person-level bootstrap confidence intervals (Section 5). On the chronological held-out test split, the model achieved an MAE of 3.178 minutes at the five-minute landmark, an all-landmark MAE of 4.185 minutes, and an all-landmark C-index of 0.621. The person-held-out all-landmark MAE was 3.951 minutes. Feature selection (Section 6) showed that a reduced 13-feature model achieved comparable performance, with a chronological all-landmark test MAE of 4.176 minutes. The user's past session length for the same app was identified as the most informative feature group, while the 18-feature model remains the reported final refinement configuration. Earlier Task-4 pilot comparisons against the personal-remaining baseline are reported separately and should not be interpreted as an independently verified improvement of the final refinement model.

Model B. Model B could not be built for its original target, because sleep-onset delay is almost always recorded as zero in GLOBEM. On Fitbit main-sleep duration, the mixed model reached a test MAE of 66.08 minutes against 66.29 minutes for the personal-mean baseline, a difference whose interval includes zero (−0.94 to +1.34 minutes). The within-person association between evening phone use and sleep duration (+0.69 minutes per additional 10 minutes of use) is observational and does not support a numerical sleep-loss warning, so Model B remains exploratory and is not used in the app.

Challenges. The main challenges were (1) that session length is only partly predictable from usage logs, which keeps all improvements in the range of seconds to a few minutes; (2) that the initial and refined Model A experiments use different samples, session definitions, landmarks and splits, so no direct before/after improvement can be claimed; (3) that long sessions, which the warning targets most, are rare in the test data (36 test snapshots at 30 minutes); and (4) that Model B's intended target turned out to be unusable.

Final status and next steps. The defensible product scope is a Model A remaining-session estimate combined with a reminder before the user's chosen bedtime, computed on the phone so that

usage histories do not leave the device. Before deployment can be claimed, the 18-feature model still needs a matching parameter export, Python–Kotlin parity tests and a working Android build (TS 5). The two-week pilot will then provide screen-event data to re-calibrate Model A for the app's own session definition and to check whether the reminders actually shorten late-night sessions.

References

Aliannejadi, M. (n.d.). LSApp: Large dataset of sequential mobile app usage [Data set]. GitHub. https://github.com/aliannejadi/LSApp

Aliannejadi, M., Zamani, H., Crestani, F., & Croft, W. B. (2021). Context-aware target apps selection and recommendation for enhancing personal mobile assistants. ACM Transactions on Information Systems, 39(3), Article 29. https://doi.org/10.1145/3447678

Android Developers. (n.d.). Notification runtime permission. https://developer.android.com/develop/ui/views/notifications/notification-permission

Android Developers. (n.d.). UsageEvents.Event. https://developer.android.com/reference/android/app/usage/UsageEvents.Event

Android Developers. (n.d.). UsageStatsManager. https://developer.android.com/reference/android/app/usage/UsageStatsManager

Android Developers. (n.d.). Develop sleep experiences with Health Connect. https://developer.android.com/health-and-fitness/health-connect/experiences/sleep

Carney, C. E., Buysse, D. J., Ancoli-Israel, S., Edinger, J. D., Krystal, A. D., Lichstein, K. L., & Morin, C. M. (2012). The Consensus Sleep Diary: Standardizing prospective sleep self-monitoring. Sleep, 35(2), 287–302. https://doi.org/10.5665/sleep.1642

Chinoy, E. D., Cuellar, J. A., Huwa, K. E., Jameson, J. T., Watson, C. H., Bessman, S. C., Hirsch, D. A., Cooper, A. D., Drummond, S. P. A., & Markwald, R. R. (2021). Performance of seven consumer sleep-tracking devices compared with polysomnography. Sleep, 44(5), zsaa291. https://doi.org/10.1093/sleep/zsaa291

Cox, D. R. (1972). Regression models and life-tables. Journal of the Royal Statistical Society: Series B (Methodological), 34(2), 187–220.

Davidson-Pilon, C. (2019). lifelines: Survival analysis in Python. Journal of Open Source Software, 4(40), 1317. https://doi.org/10.21105/joss.01317

Goldberger, A. L., Amaral, L. A. N., Glass, L., Hausdorff, J. M., Ivanov, P. C., Mark, R. G., Mietus, J. E., Moody, G. B., Peng, C.-K., & Stanley, H. E. (2000). PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. Circulation, 101(23), e215–e220. https://doi.org/10.1161/01.CIR.101.23.e215

Graf, E., Schmoor, C., Sauerbrei, W., & Schumacher, M. (1999). Assessment and comparison of prognostic classification schemes for survival data. *Statistics in Medicine, 18*(17–18), 2529–2545.

Vega, J., Li, M., Aguillera, K., Goel, N., Joshi, E., Khandekar, K., Durica, K. C., Kunta, A. R., & Low, C. A. (2021). Reproducible Analysis Pipeline for Data Streams: Open-source software to process data collected with mobile devices. Frontiers in Digital Health, 3, 769823. https://doi.org/10.3389/fdgth.2021.769823

Xu, X., Zhang, H., Sefidgar, Y., Ren, Y., Liu, X., Seo, W., Brown, J., Kuehn, K., Merrill, M., Nurius, P., Patel, S., Althoff, T., Morris, M., Riskin, E., Mankoff, J., & Dey, A. (2022). GLOBEM Dataset: Multi-Year Datasets for Longitudinal Human Behavior Modeling Generalization. Advances in

Neural Information Processing Systems 35, Datasets and Benchmarks Track. https://doi.org/10.52202/068431-1790

Xu, X., Zhang, H., Sefidgar, Y., Ren, Y., Liu, X., Seo, W., Brown, J., Kuehn, K., Merrill, M., Nurius, P., Patel, S., Althoff, T., Morris, M., Riskin, E., Mankoff, J., & Dey, A. (2023). GLOBEM Dataset: Multi-Year Datasets for Longitudinal Human Behavior Modeling Generalization (Version 1.1) [Data set]. PhysioNet. https://doi.org/10.13026/r9s1-s711